

Universidad del Valle de Guatemala

Facultad de Ingeniería · Deep Learning — Semestre 2, 2026

¿El orden de las transacciones revela fraude?

Proyecto 1 — Monitoreo transaccional

Daniel Estrada – 20853

Daniela Ramírez – 23053

1. La pregunta del comité

«Cuando revisamos los casos que se nos escaparon, el patrón siempre está ahí. No en los montos: en el orden en que ocurrieron.»

El motor actual resume cada ventana en variables agregadas: monto promedio de 24 h, transacciones por hora, monto máximo y diversidad de comercios.

El problema: todas esas variables son idénticas si se baraja la secuencia. Por construcción, el motor actual no puede ver orden.

La pregunta no es «¿puedo entrenar una LSTM?», sino **¿el orden aporta información que los agregados no capturan, y cuánto vale en quetzales?**

2. Diseño: datos donde conocemos la verdad

Generamos los datos para poder hacer una **predicción falsable**: cuatro mecanismos con dependencia del orden conocida y distinta.

Mecanismo	Orden	Predicción
escalada de prueba	fuerte	B gana
ráfaga geográfica	parcial	empate
vaciado súbito	ninguno	B NO gana
toma gradual	difuso	ambos fallan

Confusores legítimos para que no sea trivial: rachas de microcompras, compras grandes reales y viajes.

260,158 transacciones · 4,000 tarjetas · 1.07 % de fraude · $K = 20$.

Partición temporal estricta 70/15/15 por fecha. El conjunto de prueba se abrió **una sola vez**.

Vaciado súbito es el **control negativo**: si B ganara también ahí, la ventaja no vendría del orden.

3. Prueba 1 — Permutación controlada

Barajamos el orden de la historia dentro de cada ventana. Mismos eventos, mismos valores, mismas variables agregadas, y la transacción calificada se queda en su sitio: solo se destruye la secuencia.

−43 %

del AUC-PR del modelo secuencial se pierde al barajar ($0.860 \rightarrow 0.483$, cinco permutaciones distintas)

El control que no es obvio: si barajáramos también la posición de la transacción calificada, el modelo perdería además el acceso al evento que debe puntuar y la caída parecería de 90 %. Sin ese control estaríamos midiendo dos cosas y atribuyéndolas al orden.

4. Prueba 2 — Por mecanismo: el resultado incómodo

Mecanismo	A	B	B – A
escalada_prueba	0.970	0.910	-0.060
rafaga_geografica	0.921	0.909	-0.012
toma_gradual	0.837	0.645	-0.192
vaciado_subito	0.889	0.206	-0.683

Se confirmó: B se desploma en el control sin orden (0.206 frente a 0.889). Donde no hay orden, no hay nada que leer.

Se refutó: B no supera a A en ningún mecanismo. Nuestra predicción falló y así lo reportamos.

5. Entonces, ¿el orden aporta o no?

Que B pierda no significa que su señal sea **redundante**. Combinamos ambos puntajes con una mezcla ajustada **solo en validación**:

$$0.9509 \rightarrow 0.9522$$

$+0.0013 \cdot IC\ 95\ \% [+0.0001, +0.0027]$ — *no cruza cero*

Detectable $\neq$ importante. El intervalo excluye el cero, así que la señal de orden no es redundante; pero la mejora es de milésimas. Con 40,000 transacciones de prueba se detectan diferencias minúsculas, y presentar esto como «las secuencias mejoran la detección» sería exagerar lo que medimos.

6. La decisión en quetzales — y el hallazgo real

Costos del comité: **Q4,200** por fraude que pasa, **Q180** por bloqueo indebido. Asimetría 23 : 1, así que el umbral minimiza costo esperado, no F1.

Escenario	Fraudes que pasan	Bloqueos indebidos	Ahorro en prueba
Solo motor actual	17	335	Q1,787,700
Motor + señal de orden	20	240	Q1,792,200

En dinero: nada. Q4,500 de diferencia (0.25 %).
Indistinguible de no hacer nada.

En clientes: sí. 95 bloqueos indebidos menos
(28 %) a igual exhaustividad.

El caso a favor del orden es de **experiencia del cliente**, no de detección de fraude.

7. Recomendación y límites

Piloto acotado. Conservar el motor actual como decisión principal. Lo que justificaría el piloto no es detectar más fraude —eso no lo logra— sino molestar a menos clientes detectando el mismo.

Por qué B no ganó solo

Pocos fraudes etiquetados en entrenamiento (~1,900). Los agregados son conocimiento del dominio ya destilado; la red debe redescubrirlo con muy pocos positivos. Parece un límite de datos, no de arquitectura —y es comprobable.

Nuestra **apuesta C falló** (-0.087 frente al +0.02 exigido). Pre-registrada en git antes de ver el conjunto de prueba; la reportamos como quedó.

Siguiente paso: correr la permutación y la prueba de complementariedad sobre los datos reales del banco. Es una tarde de trabajo.

Límites que declaramos

- Datos sintéticos: no dicen nada del fraude real.
- Defecto de nuestro generador: sondeos y rachas legítimas se separan por monto, no solo por orden —sesga la prueba 2 a favor de A.
- La atención de C explica el caso en 14 %, frente al 60 % declarado.
- Una sola semilla por variante.